SHS

Student Health & Safety System

System design, technology, database, API, security and operations documentation

Generated 23 October 2026 | Version 1.0

Document purpose

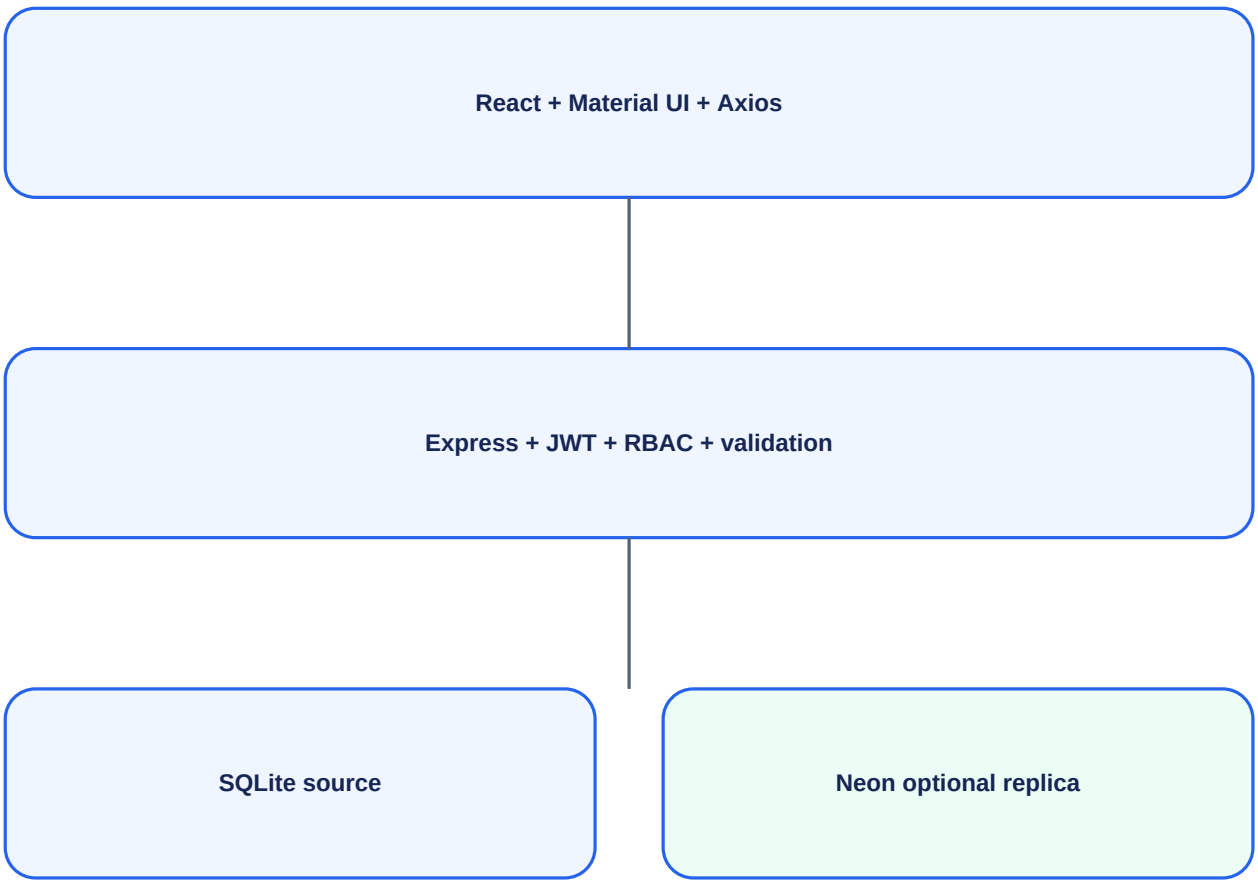
A practical technical reference for implementation, review, deployment, maintenance, and future extension.

- Student Health & Safety is a role-aware school operations system for managing student welfare, health records, safety incidents, discipline cases, alerts, staff, parents, and audit history.
- SQLite is the authoritative operational database. Express exposes a protected REST API, while React and Material UI provide the responsive browser application. A one-way optional replica synchronizes SQLite into Neon PostgreSQL without changing the application source of truth.
- The design favors simple deployment, explicit authorization, archival instead of destructive deletion, and a generic metadata-driven resource UI.

System goals and scope

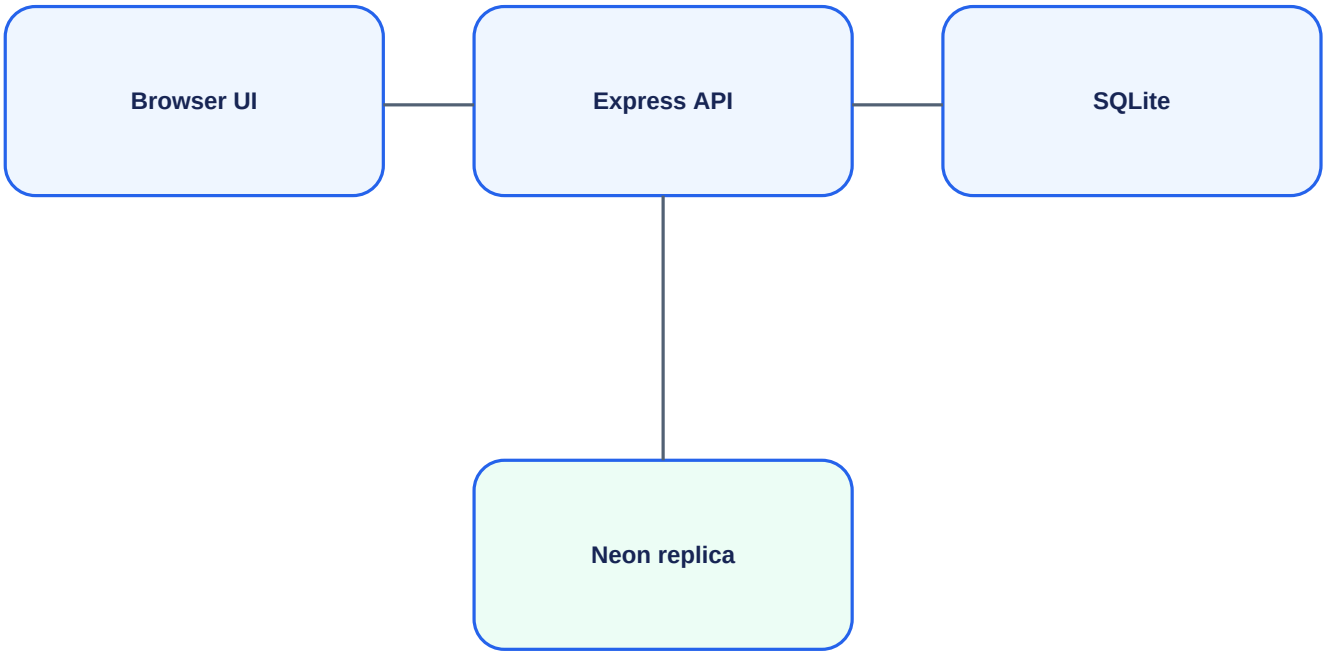
- Primary goals: centralize welfare data, reduce fragmented spreadsheets, provide a searchable record of student-related events, restrict sensitive modules by role, and provide operational summaries on the dashboard.
- In scope: authentication, role-based access control, CRUD workflows, parent linking, soft archive, audit logging, dashboard aggregates, responsive navigation, seeded relational data, and optional cloud replication.
- Out of scope: attendance automation, notifications through external messaging providers, file/document storage, automated medical diagnosis, multi-school tenancy, and a formal migration framework.

Architecture overview



- The system uses a browser-to-API architecture. The React single-page application calls Express endpoints under /api. Express authenticates requests, validates input, applies role rules, executes SQLite queries, writes audit events, and schedules optional Neon synchronization.
- Runtime layers: browser UI; Vite development server or built static assets; Express application; SQLite WAL database; optional Neon PostgreSQL replica.
- The architecture is intentionally compact: shared resource definitions drive metadata, routes, forms, filters, permissions, and table columns.

Context diagram



- Actors: administrator, staff member, medical user, and viewer. Each actor signs in and receives only the resource metadata and operations allowed by role.
- External infrastructure: the local filesystem stores the SQLite database; Neon receives a full asynchronous mirror when configured; the browser communicates over HTTP in development and should use HTTPS in production.
- Trust boundary: credentials and JWT tokens are handled by the API; the database is never accessed directly by the browser.

Technology stack

- Backend: Node.js 22.13+; Express 4; node:sqlite DatabaseSync; bcryptjs; jsonwebtoken; cors; dotenv; pg for optional Neon replication.
- Frontend: React 18; Vite 5; React Router 6; Material UI 5; @mui/icons-material; Axios; Recharts; Emotion.
- Storage and operations: SQLite in WAL mode, foreign keys enabled, environment-based configuration, npm scripts, and a static frontend build served by Express.

Repository structure

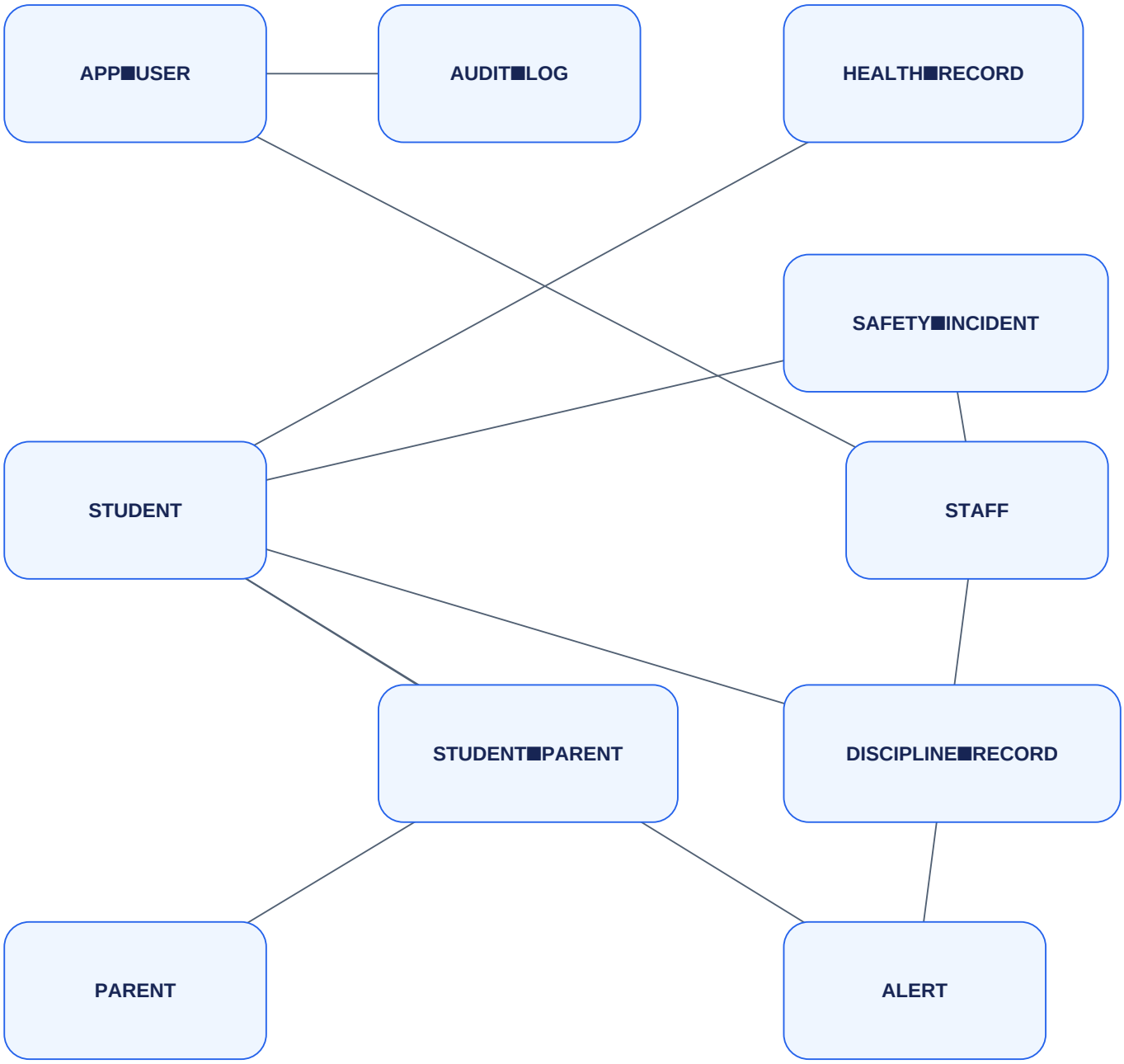
- src/db.js creates the SQLite schema and indexes. src/server.js defines authentication, metadata, dashboard, generic CRUD, relationships, and administration endpoints. src/createAdmin.js provisions the first administrator.
- src/seedData.js creates repeat-safe relational sample data. src/neonReplica.js owns the optional SQLite-to-Neon mirror.
- frontend/src/App.jsx provides the shell, navigation, authentication screen, route map, logo branding, alert badge, and responsive drawer. Pages contain the dashboard, generic resource table, detail view, and admin views.

Database design principles

- Every operational table has an archived flag and created_at/updated_at timestamps where appropriate. DELETE requests archive records rather than physically deleting them.
- Foreign keys use ON DELETE RESTRICT so related records cannot be silently removed. The student_parent table uses a composite primary key to prevent duplicate links.
- Enum-like business values are enforced with SQLite CHECK constraints for gender, severity, discipline status, alert status, and application roles.

ER diagram

The diagram on this page shows the core entities and their relationships. Student is the central subject. Parents connect through the asso



Entity catalog: students and parents

- student: student_id, name, dob, gender, class, department, phone, archived, timestamps. It is readable by all roles and writable by admin/staff.
- parent: parent_id, name, phone, email, archived, timestamps. Parents are readable by admin/staff/viewer and writable by admin/staff.
- student_parent: student_id, parent_id, relation. It models Mother, Father, or Guardian links and is managed from student detail pages.

Entity catalog: staff and users

- staff stores operational personnel: staff_id, name, role, phone, archived, timestamps. Staff is readable by all roles but writable only by administrators.
- app_user stores login identities: user_id, name, email, password_hash, role, staff_id, active, created_at. Passwords are never stored in plaintext.
- The optional staff_id relationship associates a login with a staff record without requiring every staff member to have an application account.

Entity catalog: health and incidents

- health_record stores student_id, date, illness, medication, doctor_notes, archived, and timestamps. It is sensitive and limited to admin/medical roles.
- safety_incident stores student_id, date, location, incident_type, severity, description, staff_id, archived, and timestamps.
- Incident severity is Low, Medium, High, or Critical. Incident records are readable by admin, staff, and medical users; write access is admin/staff.

Entity catalog: discipline and alerts

- discipline_record stores student_id, date, violation, action_taken, status, archived, and timestamps. Status values are Open, Under Review, Resolved, and Closed.
- alert stores student_id, staff_id, alert_type, date_time, status, archived, and timestamps. Status values are Unread, Acknowledged, and Resolved.
- Unresolved alerts are defined consistently as archived=0 and status != Resolved. The dashboard and navigation badge use this same definition.

Indexes and query paths

- Health, incident, discipline, and alert tables have student-oriented indexes for profile pages and filtered lists. Incidents and alerts also have staff indexes for staff detail views.
- student_parent has an index on parent_id for parent profiles. Composite indexes include the most common filtering or ordering fields.
- The generic list route limits page size to 100, supports pagination, validated sorting, search, filters, and date ranges.

Authentication flow

- A user submits email and password to POST /api/auth/login. The server lowercases the email, loads an active app_user, compares the password with bcrypt, and issues an eight-hour JWT.
- The frontend stores the token in sessionStorage and Axios adds it as a Bearer token. The auth middleware verifies the token and reloads the active user from SQLite.
- Logout records an audit event and clears the browser token. Token revocation is not implemented; short expiry and HTTPS are required for production hardening.

Authorization model

- The allow middleware compares the authenticated role with the resource read/write lists. Unauthorized attempts are audited as DENIED and return 403.
- Roles: admin has full access; staff manages students, parents, incidents, discipline, and alerts; medical manages health and reads incidents/alerts; viewer reads students, parents, and staff.
- Sensitive list and detail operations for health and discipline create audit entries so access to protected information is traceable.

Input validation

- Resource definitions declare required, numeric, date, enum, email, textarea, and default properties. The clean function converts empty values to null, validates fields, limits strings to 2,000 characters, and returns 422 errors.
- Database constraints remain the final integrity boundary. Foreign-key and constraint failures are translated into explicit client-facing errors rather than silently ignored.
- The frontend mirrors required-field checks in dialogs, while the backend remains authoritative.

REST API surface

- Public operational status: GET /api/status. Authentication: POST /api/auth/login and authenticated POST /api/auth/logout; GET /api/me.
- Metadata and dashboard: GET /api/meta and GET /api/dashboard. Resources: /api/students, /api/parents, /api/staff, /api/health, /api/incidents, /api/discipline, and /api/alerts.
- Administration and relationships: /api/users, /api/audit, POST/DELETE /api/links. Resource routes support GET list, GET by id, POST, PUT, and archive-style DELETE.

Generic CRUD design

- One RES object defines table, primary key, permissions, searchable columns, filters, dates, and form fields. A route loop generates consistent CRUD behavior for all modules.
- The list handler builds parameterized WHERE clauses, validates sort columns against a whitelist, applies pagination, and joins student names for student-linked resources.
- This approach reduces duplicated code while keeping business differences explicit in metadata.

Frontend shell and navigation

- App.jsx provides the fixed top bar, collapsible drawer, responsive temporary drawer on small screens, route definitions, toast notifications, quick-add menu, and unresolved-alert badge.
- Navigation entries are generated from /api/meta so users only see resources they can read. Material UI icons and the school logo provide consistent visual identity.
- The route order places administrator pages before the generic resource and detail routes.

Dashboard and analytics

- The dashboard presents counts for students, staff, incidents, health records, pending discipline, and unresolved alerts based on the signed-in role.
- Recharts renders a six-month incident trend line and a discipline status pie chart. Recent incidents, alerts, and administrator activity appear in compact tables.
- The dashboard returns counts and operational summaries, not health details. Sensitive aggregates are omitted when the role cannot read the underlying resource.

Resource tables and forms

- ResourcePage is a reusable searchable, sortable, paginated table. It derives columns, filters, date inputs, and forms from metadata.
- FormDialog supports required fields, enum select controls, dates, numbers, email inputs, password inputs, and multiline notes. Add/edit/archive actions use toast feedback.
- Archive confirmation explains that records are hidden from active lists but retained in the database.

Student detail workflow

- Student detail displays scalar profile attributes and related arrays: parents, incidents, discipline cases, alerts, and health records according to role.
- Parents can be linked and unlinked from the student page. Parent detail pages show linked students, while staff pages show incidents and alerts generated by the staff member.
- Related data is fetched by the API profile function so role checks remain server-side.

Audit and observability

- Audit entries capture login, logout, denied actions, creates, updates, archives, sensitive reads/lists, and relationship changes. Each event records user, action, entity, entity ID, detail, and timestamp.
- Operational errors are logged by the Express error handler. Client errors are surfaced through snackbar notifications.
- Recommended production additions include structured logging, request IDs, health metrics, backup monitoring, and alerting on replication failures.

SQLite operations and backups

- SQLite runs in WAL mode for improved read/write concurrency and foreign keys are explicitly enabled. The database file is data/school.db with accompanying WAL/SHM files while active.
- Back up the database only after stopping the server or using a consistent SQLite backup mechanism. Preserve the database file permissions and protect it as sensitive data.
- Schema creation is idempotent CREATE TABLE IF NOT EXISTS logic. There is currently no versioned migration system.

Neon replica design

- Neon replication is optional and one-way. SQLite remains authoritative. On startup and after successful writes, a scheduled full mirror creates the PostgreSQL-compatible schema, truncates replica tables, and copies current rows.
- NEON_DATABASE_URL enables the feature. If absent, the application logs that replication is disabled and continues normally.
- The mirror is asynchronous and failures are logged without changing the successful SQLite response. Production deployments should add retry/backoff and replication lag monitoring.

Deployment, security, testing and roadmap

- Development: run npm install, configure .env, create an administrator, seed data if desired, start the backend, and run the frontend Vite server separately.
- Production: build the frontend, serve the generated assets through Express, set a strong JWT_SECRET, use HTTPS, restrict CORS, protect .env, and run the Node process under a supervisor.
- Key settings: PORT, JWT_SECRET, DB_FILE, CORS_ORIGIN, ADMIN_EMAIL, ADMIN_NAME, ADMIN_PASSWORD for provisioning, and optional NEON_DATABASE_URL.

Operational runbook

- Startup checklist: verify Node version; verify .env; ensure database directory exists; run create-admin if needed; start the server; check GET /api/status; check frontend login.
- Incident checklist: inspect server logs; verify SQLite file and WAL; test the affected API route; inspect audit_log; check Neon replica errors; preserve a backup before repairs.
- Data-change checklist: prefer UI/API writes; do not edit SQLite manually while the server is running; confirm foreign-key references; run a post-change count and replica verification.

Appendix: route and role matrix

- Students: read admin/staff/medical/viewer; write admin/staff. Parents: read admin/staff/viewer; write admin/staff. Staff: read all; write admin.
- Health: read/write admin/medical. Incidents: read admin/staff/medical; write admin/staff. Discipline: read/write admin/staff.
- Alerts: read admin/staff/medical; write admin/staff. Users and audit are administrator-only. Parent links are admin/staff.

Appendix: glossary and conclusion

- CRUD means create, read, update, and archive/delete behavior. WAL means write-ahead logging. JWT means JSON Web Token. ER means entity-relationship. RBAC means role-based access control.
- The system is a compact, coherent foundation for school welfare operations. Its strongest design choices are a clear relational model, centralized permissions, soft deletion, an explicit audit trail, and a UI generated from backend metadata.
- For delivery, keep SQLite as the source of truth until operational requirements justify a primary PostgreSQL migration. Treat Neon replication as a resilience and reporting aid, not as a second writer.